

COMP90041

Programming and Software Development

Lecture 7 - Object Oriented Programming

Danny Xu

The University of Melbourne acknowledges the Traditional Owners of the unceded land on which we work, learn and live: the Wurundjeri Woi-wurrung and Bunurong peoples (Burnley, Fishermans Bend, Parkville, Southbank and Werribee campuses), the Yorta Yorta Nation (Dookie and Shepparton campuses), and the Dja Dja Wurrung people (Creswick campus).

The University also acknowledges and is grateful to the Traditional Owners, Elders and Knowledge Holders of all Indigenous nations and clans who have been instrumental in our reconciliation journey.

We recognise the unique place held by Aboriginal and Torres Strait Islander peoples as the original owners and custodians of the lands and waterways across the Australian continent, with histories of continuous connection dating back more than 60,000 years. We also acknowledge their enduring cultural practices of caring for Country.

We pay respect to Elders past, present and future, and acknowledge the importance of Indigenous knowledge in the Academy. As a community of researchers, teachers, professional staff and students we are privileged to work and learn every day with Indigenous colleagues and partners.

WARNING

This material has been reproduced and communicated to you by or on behalf of the University of Melbourne in accordance with section 113P of the *Copyright Act 1968 (Act)*.

The material in this communication may be subject to copyright under the Act.

Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act.

Do not remove this notice

Overview



In this part of the lecture, you will learn about:

- **Classes Contd.**
 - Packages
 - Math Class
 - Javadoc
- **Basics of Object-Oriented Programming**
- **Inheritance and Access Modifiers**
- **Polymorphism and Abstract Classes**



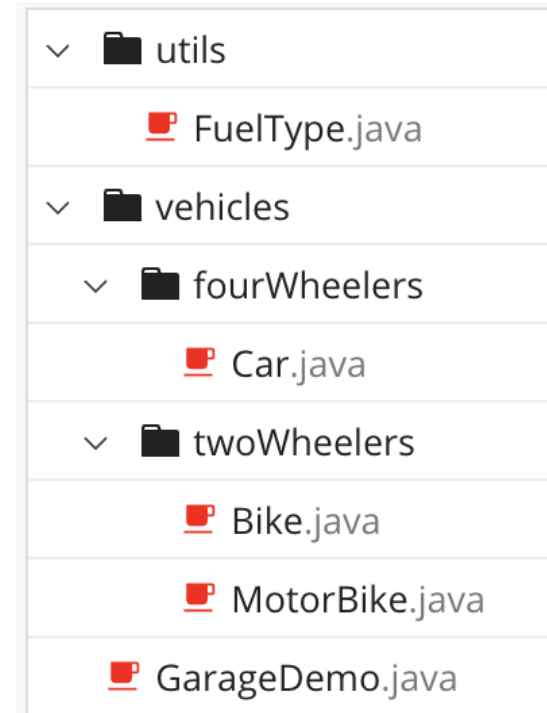
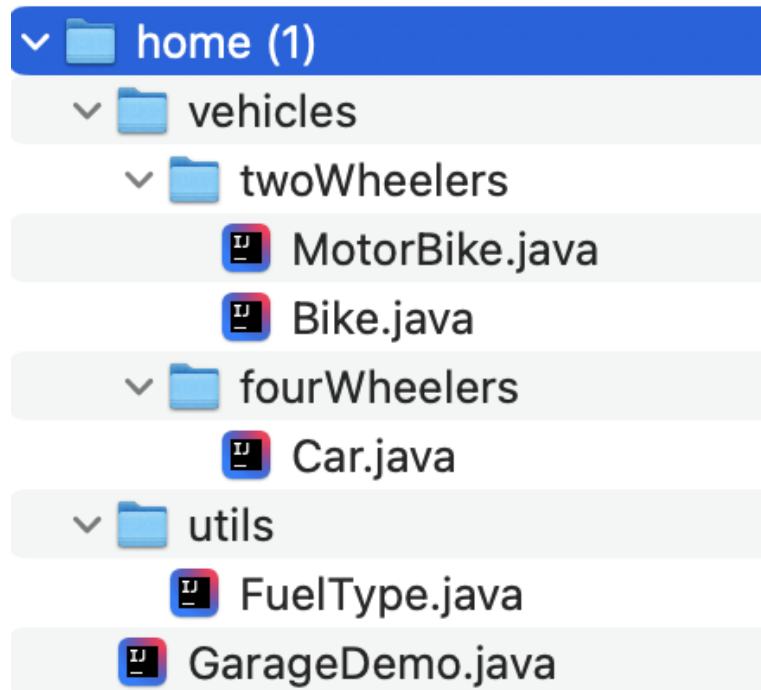
THE UNIVERSITY OF
MELBOURNE

Classes Contd.

Packages



- A package is a **group of classes**.
- Packages can have a group of packages nested inside as well.
- Why Packages? Do you have nested folders in your laptops? Don't want to dump everything together - that's called Garbage Bin!!



When and why to create packages



- Too many classes present in your main directory.
- Classes that are relatable should go together in a package.
- Improved searchability. Able to locate similar files together.
- Helps with name clashes.
 - For example – Constants.java contains car related constants, another Constants.java contains fuel related constants. Both can reside in different packages and co-exist.
 - Files accessed using package name helps to resolve clashes.

How to create and use packages?



- Create a file under a package. Define using **package** statements.
 - Package statement has to be the first statement in the file.
 - If the file is not in a folder, means no package and no package statement.
- Use in another file in a different package using **import** statement.
 - If the

```
package utils;  
public enum FuelType{  
    PETROL,  
    DIESEL,  
    ELECTRIC;  
}
```

utils/FuelType.java

```
package vehicles;  
  
import utils.FuelType;  
  
public class Car {  
    String carName;  
    FuelType fuelType;  
}
```

vehicles/Car.java

A final word about Packages



- Use import statements reasonably. No `import packagename.*;`
 - Worse readability of code due to lack of info of which package is used
 - Possibly longer compilation time
 - Larger possibility of conflict of same package names with other packages
 - Possibility of conflict of same class names residing in other packages.
- Classes that are not present in any package are present in `default` package.
- **CLASSPATH** – directory where Java compiler and runtime looks for packages
 - You can set CLASSPATH to any folder as well.
 - Current directory is included if no CLASSPATH mentioned.
 - Usually starts from the directory where the file with main method is present.
- `java.lang` is automatically imported and that's how you use `System.out.println`, `String` or `Math`

Math Class



- Provides Math related methods and constants
 - Methods - Sin, cos, tan, sqrt, abs, pow, log etc.
 - Constants – $\text{PI} = 3.14\dots$, $\text{EXP} = 2.718\dots$
- Overloaded methods
 - Absolute value method - `abs (double)`, `abs (int)`, `abs(float)` etc.
 - Round method – `round (double)`, `round (float)`
- Some interesting methods – round
 - Round up if above 0.5, else down. `round (3.56184, 2) = 4`, `round (3.26) = 3`
 - Round up to next integer `ceil(3.26184) = 4`
 - Round down to next integer `floor (3.86) = 3`

Java Commenting Styles



- Single line comments

`// this is a single line comment in java`

`// this can be used for multi-line, each line uses //`

- Multi line comments

`/*this is a multi-line comment in java`

`this goes on in multi-line, using a / & *`

`No need for symbols on each line */`

- Java doc comments

`/** this is a javadoc style comment`

`Uses one / and two *`

`but this isn't free style commenting **/`

Javadoc Comments



- Creates document to explain the classes and methods written
 - What are the methods, instance variables, constants in the class?
 - What does these methods do?
 - What arguments these methods take?
 - What are the data types of these arguments?
 - What are the return types of these methods?
 - What are the constants and their values?
- `javadoc` command run to generate HTML files out of java files if proper javadoc comments present.
- The document generated is called **API (Application Programming Interface)**



THE UNIVERSITY OF
MELBOURNE

Object Oriented Programming

Programming Paradigms



There are many many programming paradigms. Typically,

- Procedural Programming – focus on **procedures/functions**, e.g., C, Pascal, Basic
- Object Oriented Programming – focus on creating **objects** from **classes** that resembles **real-world entities**, groups data and methods together, e.g., Java, C++, C#, Python but OOP style
- Functional Programming – focus on **mathematical functions** and **immutability**, e.g., Scala, Lisp
- Declarative Programming – uses formal **logic** to express **facts** and **rules and things to do**, e.g., Prolog
- And many more....

Procedural vs Object Oriented Programming



Feature	Procedural Programming	Object-Oriented Programming (OOP)
Approach	Follows a top-down, step-by-step approach	Follows a bottom-up, modular approach
Focus	Focuses on functions and logic	Focuses on objects and data
Data Handling	Data is exposed and can be freely accessed	Data is encapsulated and accessed via methods
Code Reusability	Limited, requires code duplication	Promotes reuse through inheritance and classes
Examples of Languages	C, Pascal	Java, C++, Python (OOP style)
Modularity	Uses functions for modularity	Uses objects/classes for modular design
Security	Less secure, as data can be modified globally	More secure via encapsulation and access control
Real-World Modeling	Not natural for modeling real-world entities	Easily models real-world systems using objects

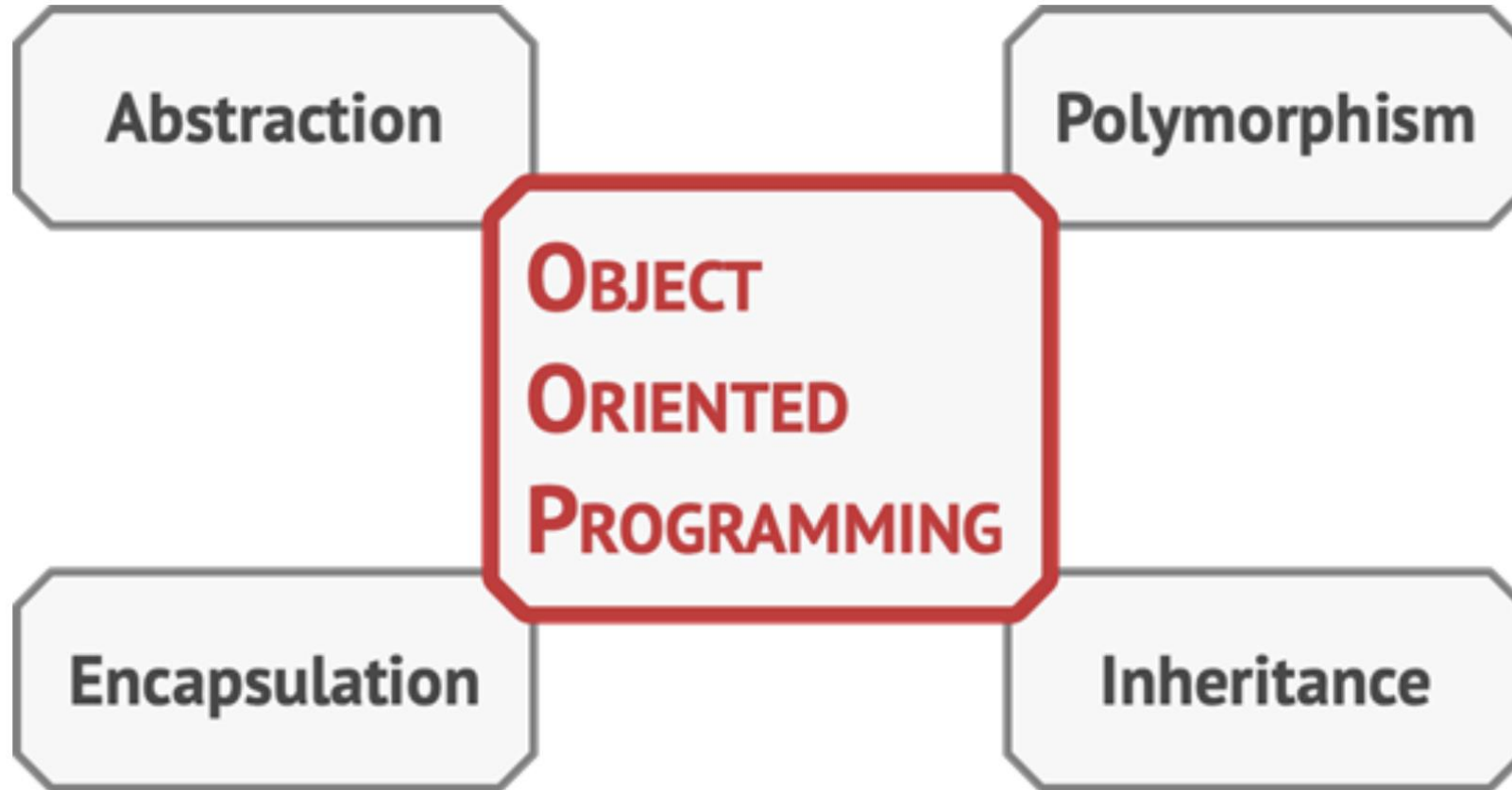


Object Oriented Programming (OOP)



- Everything should be an object.
- Objects come from classes.
- Class? Wrap data properties and behaviour/actions related to them in one place.
- Class resembles **real-world entities**. Also called **blueprints**.
- Create many many classes to resolve a problem statement.
- Create many many objects from classes. The actual data may differ in objects but all objects of a class have same data properties/data types/behaviours/actions.
- Data properties in classes are also called **fields**. The actions called **methods**.

Pillars of OOP

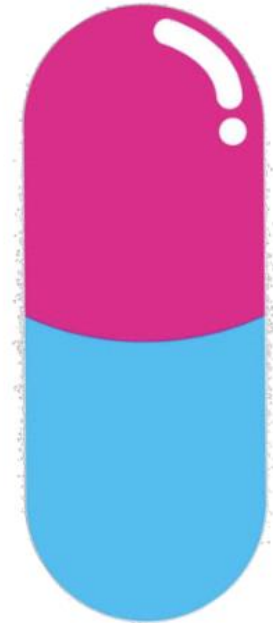


Encapsulation

- Imagine a Kitchen. You have a stove to cook, an oven to bake, a dishwasher to wash dishes, different cabinets to store things.
- It makes sense to keep the attributes of the kitchen like stove, oven, dishwasher and their behaviour like cook, bake, wash together at one place. Wrap them in a capsule aka "class" named Kitchen.
- "Encapsulate" data properties and actions/behaviours of the data properties in a class.

Attributes are
the pink side

Methods are
the blue side



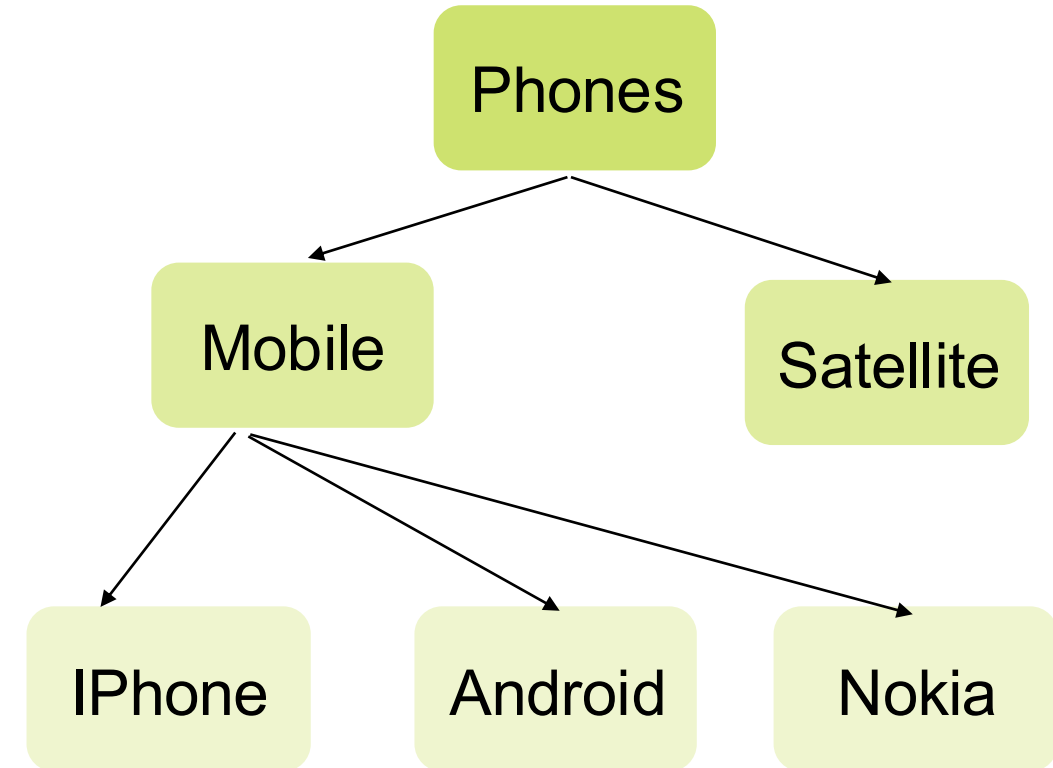
Abstraction

- Love Travelling? Book a Ticket, board a plane, fly to the destination.
- Do we know what's inside an Airplane? No, not relevant to us.
- Abstraction is an art of hiding unnecessary or complex details from the outside world.
- How do we do it? Make things private.



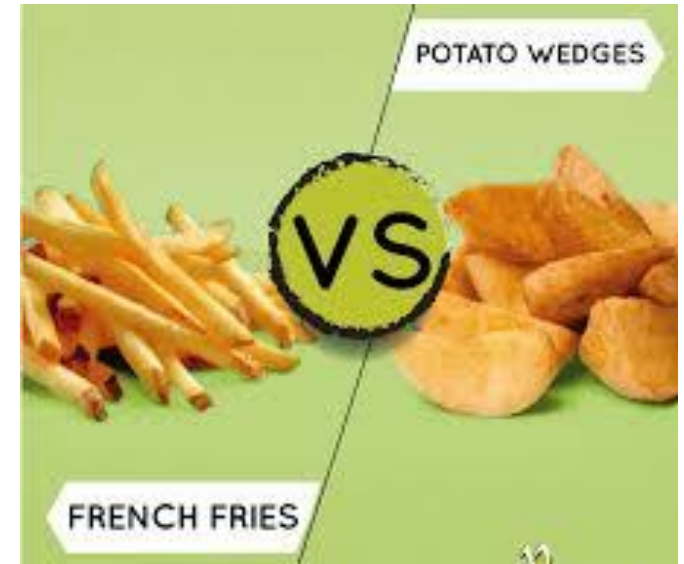
Inheritance

- Mobile Phones – iPhone, Android, Nokia, Satellite Phones.
- How many classes should we create? 4 perhaps.
- What is the common functionality? They all make calls.
- How many methods do we write? Perhaps 4 in 4 different classes? **NO**
- Reduce code duplication by writing common data/actions once and inherit from the parent.



Polymorphism

- Poly means **many**, morph means **form**. Usually applies to methods.
- French fries vs potato wedges?
- Why would we need polymorphism?
 - Same/similar functionality different data types. Overloading? Eg -
 - `abs(float a)`
 - `abs(int a)`
 - `abs(double a)`
 - Similar kind of object but different functionality. Eg -
 - Brakes in a bicycle
 - Brakes in car
 - Brakes in a train



Additional Concepts : Coupling & Cohesion



- Cohesion refers to sticking things together within a class that are relevant. And keeping things separate when they are not related.
- Coupling is how related or dependent two or more classes are towards each other.
- Example - Imagine a coffee machine that has
 - A heating unit
 - A coffee beans grinder
 - A brewer

Cohesion Example



- High cohesion means coffee machines heats water/milk, grinds coffee, brews coffee, may mix with milk and serve a coffee.
- Low cohesion means coffee machine also has a mixer/blender where you can also mix smoothies.
- You just mixed smoothie maker and coffee maker in one machine.
- Does that help? Some people may say yes, for someone having such machine at home, it is versatile. For large cafes, No. Because while you are waiting for coffee to brew, you can't use mixer. Or may be some other issues.
- Remember, you are not writing softwares for one person. You are writing it for a large group. Keep things separate, keep it clean.

Coupling Example



- Low coupling is when the grinder works separately than the heating unit.
- If heating units get damaged or needs to be replaced with a high-power heating unit, the grinder is not affected or doesn't need to be replaced.
- If heater and grinder are tightly coupled, a change in one, may affect other.

What do we want?



High Cohesion,

Low Coupling!!!

Inheritance



THE UNIVERSITY OF
MELBOURNE

Base Classes vs Sub Classes



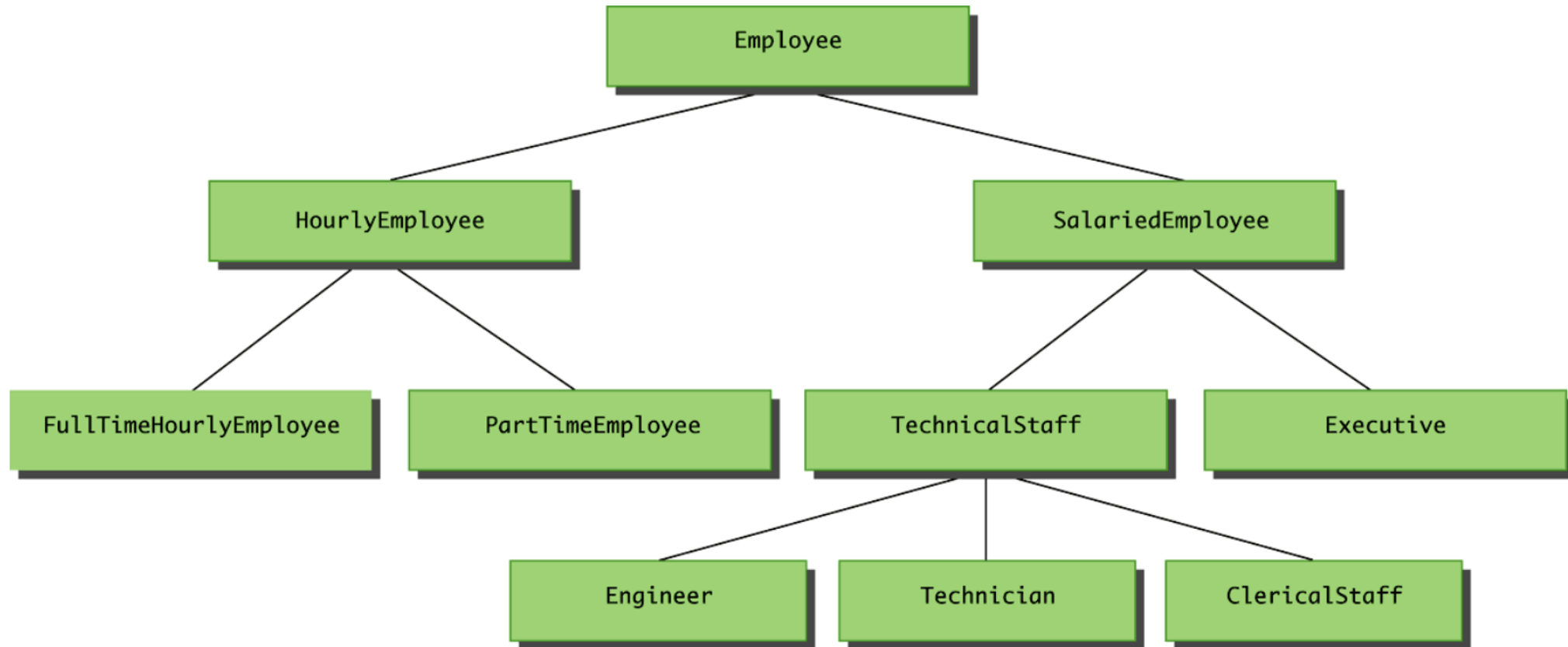
- Inheritance is a process where we create new classes from a parent class.
- We want to inherit properties from the parent class. Example – your parent and you may have similar looking eyes, nose. You inherited their features.
- Sometimes you want to modify some of the properties because a child can be slightly different than the parent.
- The new class is called a **derived/child/sub class**.
- The original class is called the **base/parent/super class**.

Why do we want to do this?



- Code reusability. Define once, reuse everywhere.
- Differentiate amongst siblings. Override the behaviours (methods) from parents.
- Add additional data properties that the parents didn't have but the child class have.
- Implement the same behaviour but differently in child classes.

Example



Code Example

```
public class Employee{  
  
    int empCode;  
    String fullName;  
  
    public void displayDetails{  
        System.out.print(empCode + ":" + fullName);  
    }  
}
```

- Inherits empCode and fullName and reuse
- Overrides displayDetails from parent

```
public class CasualEmployee extends Employee{  
  
    float hourlyRate;  
  
    public float calculateSalary(int numHours){  
        return hourlyRate * numHours;  
    }  
    @Override  
    public void displayDetails(){  
        System.out.print("Casual : " + empCode + ":" + fullName);  
    }  
}
```

Additional
properties

Different
behaviours

```
public class SalariedEmployee extends Employee{  
  
    float annualSalary;  
  
    public float calculateSalary(){  
        return annualSalary / 12;  
    }  
}
```

Access Modifiers for Inheritance



- What do we inherit from parent?
- We can inherit data variables, constants, methods -
 - Anything that is public
 - Anything that is protected
 - Anything that has no access modifier (also called default modifier), if within same package
- We don't inherit
 - **private members from parent.**
 - **Constructors are not inherited but are called in a hierarchical order.**

Just use super



- How do we access things inherited from parent?
 - `super()` to call the constructors // add arguments if parameterised
 - `super.variableNames`
 - `super.methodNames()` // add arguments if parameterised
 - `this.variableNames` // bad practice but not incorrect
 - `this.methodNames()` // bad practice but not incorrect
 - Call directly using `variableName` or `methodNames` // bad practice
- If something belongs to parent class, use **super** keyword for better readability than using **this** keyword.
- Helps to differentiate what is present in current class vs what is inherited from parent.

Constructors



- Constructors are not inherited but are called in hierarchical order starting from topmost parent to child.
- If no explicit `super()` called, the default constructor of parent still gets invoked.
- Eg – Class A is inherited by Class B that is further inherited by Class C

```
public class A{  
    A(){  
        System.out.println("In class A");  
    }  
}
```

```
public class B extends A{  
    B(){  
        System.out.println("In class B");  
    }  
}
```

```
public class C extends B{  
    C(){  
        System.out.println("In class C");  
    }  
}
```

```
public static void main(String[] args){  
    C objC = new C();  
}
```

Output:

```
In class A  
In class B  
In class C
```

Constructors



- How to call parameterised constructor of parents?
- First invoke `super()` with parameters and then update other things.
- Example employee

```
public class Employee{
    int empCode;
    String fullName;

    Employee(int Code, String name){
        this.empCode= code;
        this.fullName = name;
    }
}

public class CasualEmployee extends Employee{
    float hourlyRate;

    CasualEmployee(int code, String name, float rate){
        super(code, name);
        this.hourlyRate = rate;
    }
}
```

```
public class Tutor extends CasualEmployee{
    String subjectName;
    String supervisor;

    Tutor (int code, String name, float rate, String subject, String sup){
        super(code, name, rate);
        this.subjectName= name;
        this.supervisor = sup;
    }
}
```

```
public static void main(String[] args){
    Tutor tutor= new Tutor(123,"Nat",100.00, "COMP90041", "Trina");
}
```

Overriding a Method



- Use `@Override` at the top of the method to be overridden.
- Use same method definition as the base class.
 - `@Override` is meaningless if the method definition in base class is different than child class.

```
public class Employee{  
  
    int empCode;  
    String fullName;  
  
    public void displayDetails{  
        System.out.print(empCode + ":" + fullName);  
    }  
}
```

```
public class CasualEmployee extends Employee{  
  
    float hourlyRate;  
  
    public float calculateSalary(int numHours){  
        return hourlyRate * numHours;  
    }  
  
    @Override  
    public void displayDetails(){  
        System.out.print("Casual : " + empCode + ":" + fullName);  
    }  
}
```

Overriding a Method



- Can we change the access modifier of overridden method? **Yes, we can.** But they must be at least as accessible as method in the base class.
 - Base class has protected access, child class has public access – can be done
 - Base class has public access, child class has protected access – **cannot** be done
 - Base class has public/protected access, child class has private access – **cannot** be done.
- Java compiler will throw you an error **attempting to assign weaker access privileges** if child class has lesser/weaker access.

Overriding a Method



- Can we change the return type of the overridden method?
 - No, if the return type is a primitive
 - **Yes, if return type is a class.** But they must be a direct descendant of the type used in BaseClass. Also called **covariant** type.

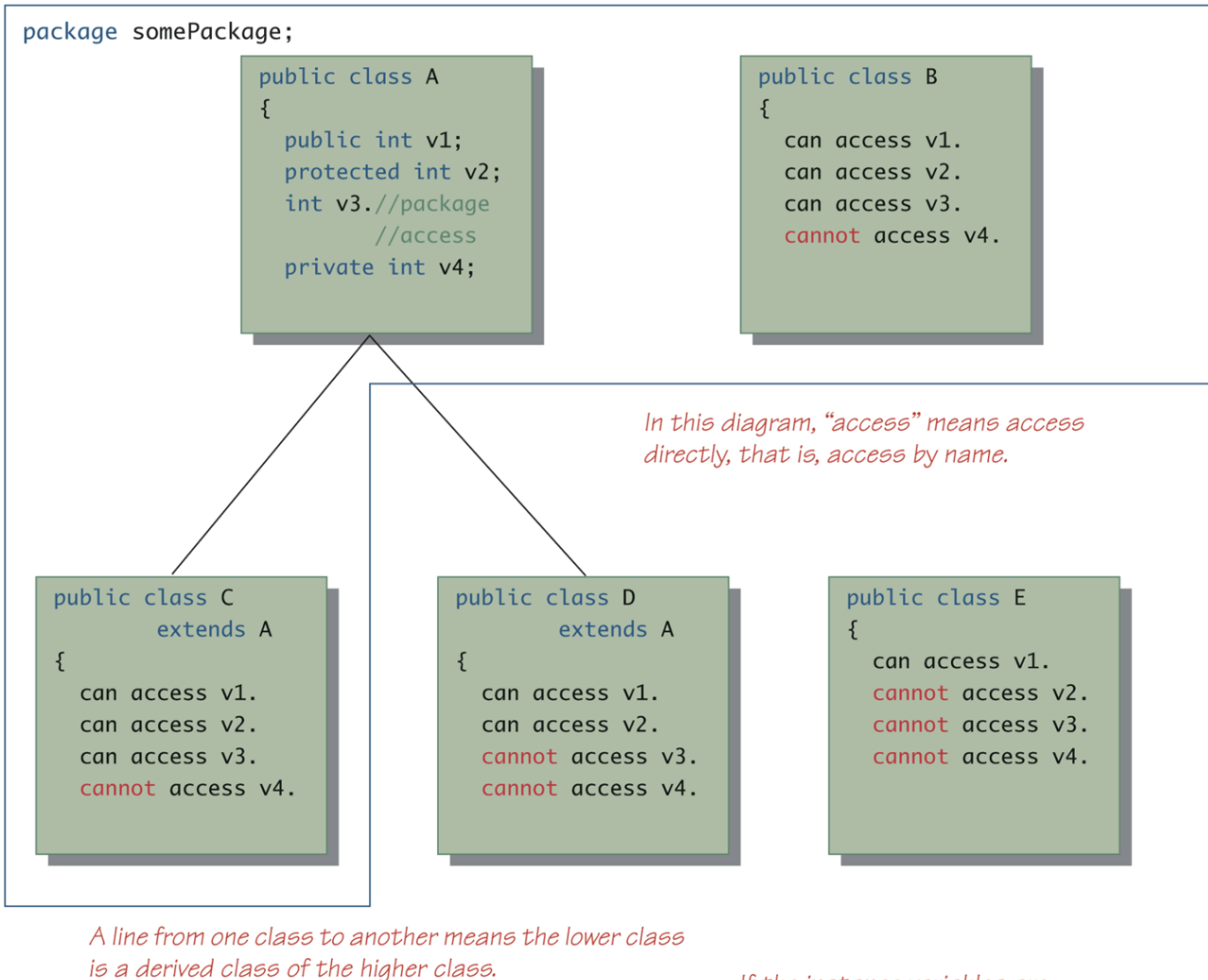
```
1 public class Employee{
2     int empCode;
3 }
4
5 public class HourlyEmployee extends Employee{
6     float hourlyRate;
7 }
8
9 public class BaseClass {
10     public Employee getSomeone (int someKey) {}
11 }
12
13 public class DerivedClass extends BaseClass {
14     public HourlyEmployee getSomeone(int SomeKey) {}
15 }
```

Preventing Inheritance



- How to stop anything from changing?
- Remember variables change, constants don't. Constants use `final`
- Inheritance allows inheriting and overriding.
- Stop allowing change/overriding by using `final`
- `final` in front of a method stops the child classes from overriding the implementation of parent class.
- `final` in front of a class stops any other classes extending/inheriting it.

Protected and default access modifiers



Is-a vs has-a relationships



- Noun denotes classes or data properties in a class.
- When you see "is-a" relationship, think Inheritance. Example – a tutor is a casual employee.
- When you see "has-a" relationship, think instance variables. Example – a tutor has a name and employee code.
- Sometimes a complex class can have an instance variable referring to another class. But it will still have "has-a" relationship. Example - Say Date is a class. A tutor has a joining Date. You can create a Date joiningDate; as an instance variable in a Tutor class.

Object class – Superman Indeed!



- Every class in Java is inherited from `Object` class.
- It's like Air. You don't see it but it is there.
- Have you noticed `@Override` on top of `toString()` and `equals()` methods in Classes?
- Why are we overriding, who are we inheriting from?
- No `extends` keyword used but the JAVA runtime knows every class inherits `Object` class and hence inherits the `toString()` and `equals()` methods.

instanceOf vs getClass()



- Every class inherits from `Object` class and gets `getClass()` method.
- `getClass()` when invoked on an object using `object.getClass()` returns the class that the object belongs to.
- `instanceOf` checks if an object belongs to a class and returns true or false.
- Example -

```
public class Main{  
  
    public static void main(String[] args){  
        Employee emp = new Employee();  
        System.out.println(emp.getClass());  
        System.out.println(emp instanceof Employee);  
    }  
}
```

Output:
Employee
true

Implementing equals in Classes



```
public boolean equals(Object otherObject)
{
    if(otherObject == null)
        return false;
    else if(getClass( ) != otherObject.getClass( ))
        return false;
    else {
        Employee otherEmployee = (Employee)otherObject;
        return (name.equals(otherEmployee.name) &&
            hireDate.equals(otherEmployee.hireDate));
    }
}
```

Check class of the object, if not equal, return false

If equals, cast the object to a class, use properties

Pitfalls



- Remember about the default package and protected packages. Privacy leaks happen quite easily if you put other classes in same package.
- Only put related classes or classes that follow inheritance hierarchy in a package, if using protected members.
- If inherited classes are in different package, they can still access members from parents if they are protected. But if you create an instance of an object in child class, cannot access protected members directly.

Example

```
package entities;
public class Employee{

    protected int empCode;
    protected String fullName;

    public void displayDetails(){
        System.out.print(empCode + ":" + fullName);
    }
}
```

```
package otherEntities;
import entities.Employee;
public class CasualEmployee extends Employee{

    float hourlyRate;

    @Override
    public void displayDetails(){
        System.out.print("Casual : " + empCode + ":" + fullName);
    }

    public void otherDetails(){
        Employee emp = new Employee();
        emp.fullName = "John Doe";
    }
}
```

Throws error,
Cant access protected in
different package



THE UNIVERSITY OF
MELBOURNE

Polymorphism and Abstract Classes

Overloading vs Overriding



- Poly means **many**, morph means **form**. Usually applies to methods.
- Why would we need polymorphism?
 - **Overloading** – Same/similar functionality different data types. . Eg -
 - `abs(float a)`
 - `abs(int a)`
 - `abs(double a)`
 - **Overriding** - Similar kind of object but different functionality. Eg -
 - `displayDetails()` of Employee
 - `displayDetails()` of CasualEmployee

Overloading vs Overriding



Feature	Method Overloading	Method Overriding
Definition	Same method name, different parameters	Same method name and parameters, in a subclass
Where It Happens	Within the same class	Between superclass and subclass
Purpose	To perform similar tasks in different ways	To provide a specific implementation in subclass
Polymorphism Type	Compile-time (Static)	Runtime (Dynamic)
Parameters	Must be different (type, number, or order)	Must be exactly the same
Return Type	Can be different	Must be same (or covariant)
Inheritance Needed?	 Not required	 Required (uses inheritance)
Example Analogy	 A tool with multiple settings for different uses	 Replacing a base scene in a play with a new one

Early Binding vs Late Binding



- **Early/Static/Compile-Time** Binding – bind a method at compile time.
- How does the compiler know? Looks at the method parameters.

```
class Calculator
{
    int add(int a, int b) {
        System.out.println("integer addition");
        return a + b;
    }

    double add(double a, double b) {
        System.out.println("double addition");
        return a + b;
    }

    public static void main(String[] args){
        Calculator calc = new Calculator();
        calc.add(3.0, 4.0); // knows to call double add because of double params
        calc.add(3,4); // knows to call integer add because of integer params
    }
}
```

Early Binding vs Late Binding



- **Late/Dynamic/RunTime** Binding – bind a method at runtime.
- Compiler doesn't know. Because params looks the same. Need to create object first.

```
class Animal {  
    void speak() {  
        System.out.println("Animal speaks");  
    }  
}  
  
class Dog extends Animal {  
    void speak() {  
        System.out.println("Dog barks");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Animal a = new Dog(); //but a is of type Animal.  
        a.speak(); // Does not know to call Animal speak or Dog Speak  
    }  
}
```

Output:
Dog barks

UpCasting vs DownCasting



- Upcasting is when an object of a derived class is assigned to a variable of a base class (or any ancestor class).
- Downcasting is when a type cast is performed from a base class to a derived class (or from any ancestor class to any descendent class)

```
public static void main(String[] args){  
    Employee emp ;  
    CasualEmployee casual = new CasualEmployee();  
    emp = casual;  
}
```

Upcasting

```
public static void main(String[] args){  
    CasualEmployee casual ;  
    Employee emp= new Employee();  
    casual = emp; // results in compile time error  
}
```

Downcasting

- Remember downcasting double to int gives a compile time error – possible lossy conversion, because there is a data loss. Similarly when you downcast like this, casual now points to Employee that doesn't have the instance variable hourlyRate, So data is lost.
- Downcasting like `casual = (CasualEmployee) emp;` may lead to runtime error.

Abstract Art ? No, Abstract Classes!



- Classes that cannot define some methods concretely are called Abstract Classes.
- Methods that are not defined completely are called abstract methods.
- Uses the keyword "abstract"
- Why? Because we don't know how will they behave. No one way to define.
 - Different sub classes may have different implementation.
 - Example – calculateSalary – different for Salaried person vs different for Casual staff.
 - Previous example had methods in subclasses, and it worked fine. Why we need abstract?
 - Because we want to enforce developers to define that behaviour/method for every subclass.

Previous Example

```
public class Employee{  
  
    int empCode;  
    String fullName;  
  
    public void displayDetails{  
        System.out.print(empCode + ":" + fullName);  
    }  
}
```

- Inherits empCode and fullName and reuse
- Overrides displayDetails from parent

```
public class CasualEmployee extends Employee{  
  
    float hourlyRate;  
  
    public float calculateSalary(int numHours){  
        return hourlyRate * numHours;  
    }  
    @Override  
    public void displayDetails(){  
        System.out.print("Casual : " + empCode + ":" + fullName);  
    }  
}
```

Additional
properties

Different
behaviours

```
public class SalariedEmployee extends Employee{  
  
    float annualSalary;  
  
    public float calculateSalary(){  
        return annualSalary / 12;  
    }  
}
```

Problem?

A class may choose not to define the method at all. Leads to inconsistent behaviour

```
public class CasualEmployee extends Employee{  
  
    float hourlyRate;  
  
    public float calculateSalary(int numHours){  
        return hourlyRate * numHours;  
    }  
    @Override  
    public void displayDetails(){  
        System.out.print("Casual : " + empCode + ":" + fullName);  
    }  
}
```

```
public class FullTimeEmployee extends Employee{  
  
    float annualSalary;  
  
    public float calculateSalary(){  
        return annualSalary / 12;  
    }  
}
```

```
public class PartTimeEmployee extends  
Employee{  
  
    @Override  
    public void displayDetails(){  
        return annualSalary / 12;  
    }  
}
```

Didn't define
CalculateSalary
method



Solution?



- Make the method abstract.
- When a method is abstract, it makes the class abstract.
- You cannot create instances or objects of Abstract classes.
- If you cannot create objects, you cannot use them. Thus enforcing a consistent behaviour!
- If you want to use them, implement the abstract methods in sub classes.
- All subclasses must implement all the abstract methods inherited from all the parents (grandparents)
- Or be declared as abstract class themselves.

Solution



```
public abstract class Employee{  
    int empCode;  
    String fullName;  
  
    public abstract float calculateSalary();  
}
```

Or make the subclass
Abstract

Either implement the
abstract method

```
public abstract class CasualEmp extends  
Employee{  
    float hourlyRate;  
}
```

```
public class Tutor extends CasualEmp{  
  
    public float calculateSalary(int numHours){  
        return hourlyRate * numHours;  
    }  
}
```

```
public class SalariedEmp extends  
Employee{  
  
    float annualSalary;  
  
    public float calculateSalary(){  
        return annualSalary / 12;  
    }  
}
```

```
public class PartTimeEmp extends Employee{  
  
    float annualSalary;  
    float fraction;  
  
    public float calculateSalary(){  
        return annualSalary * fraction / 12;  
    }  
}
```



THE UNIVERSITY OF
MELBOURNE

Live Coding (class inheritance)

Live Coding – Employees!!



- Let's build the Employee example. Employee , Casual Employee, Tutor, Salaried Employee, Academic Staff, Non-Academic Staff.
- We will look at syntax of below concepts
 - How and when to create super class/sub classes.
 - Access modifiers and pitfalls
 - Nested levels of inheritance
 - Abstract Classes and abstract methods
 - Overriding vs Overloading

Additional Reading Resources



- WALTER, S. Absolute Java, Global Edition. [Harlow]: Pearson, 2016. (Chapter 7 and 8)
- SCHILDT, H. Java: The Complete Reference, 12th Edition: McGraw-Hill, 2022 (Chapter 8 and 9)
- Object Oriented Programming Concepts (accessible on 14-02-2024) The Java Tutorials. Available at:
<https://docs.oracle.com/javase/tutorial/java/concepts/index.html>
- Shvets, A. Dive Into DESIGN PATTERNS.

(c) The University of Melbourne 2023

This material is protected by copyright. Unless indicated otherwise, the University of Melbourne owns the copyright subsisting in the work. Other than for the purposes permitted under the Copyright Act 1968, you are prohibited from downloading, republishing, retransmitting, reproducing or otherwise using any of the materials included in the work as standalone files. Sharing University teaching materials with third-parties, including uploading lecture notes, slides or recordings to websites constitutes academic misconduct and may be subject to penalties. For more information: [Academic Integrity at the University of Melbourne](#)

Requests and enquiries concerning reproduction and rights should be addressed to the University Copyright Office, The University of Melbourne: copyright-office@unimelb.edu.au

The University of Melbourne (Australian University)
PRV12150/CRICOS 00116K

See you next time!



THE UNIVERSITY OF
MELBOURNE